SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	40% of CIA	**Total Marks:**	2 * 50 = 100 (2 Question)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	A. Mounish	**Reg. No.:**	192472273
Section / Batch:	CSE AI	**Date:**	10/08/26

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.* Signature of the Student: _____

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

A. Basic API Integration and Text Generation

9. AI Resume Information Extractor: Accept resume text and extract candidate name, skills, education, experience, and certifications in structured JSON format.

API	Language	Output
Hugging Face	Python	JSON

1. Problem Statement

Develop an AI Resume Information Extractor that accepts resume text as input and extracts the candidate name, skills, education, experience, and certifications in structured JSON format using an AI API.

2. Aim

To develop an AI-based Resume Information Extractor using Python and the Hugging Face Inference API. The system converts unstructured resume text into organized JSON data that can be easily viewed, stored, or processed by another application.

3. Objectives

- Accept multiline resume text from the user.
- Connect the Python application to an AI API.
- Extract candidate name, skills, education, experience, and certifications.
- Return the extracted information in valid JSON format.
- Handle empty input and API errors without crashing.

4. Technologies Used

- Python
- Visual Studio Code
- Hugging Face Inference API
- OpenAI-compatible Python client
- AI model: openai/gpt-oss-120b:fastest
- JSON for structured output

5. System Workflow

Resume Text → Python Program → Hugging Face API → AI Model → Information Extraction → JSON Output

6. Implementation in VS Code

1. Create a project folder named AI_Resume_Extractor and open it in VS Code.

2. Create and activate a Python virtual environment.

3. Install the required library using: pip install openai

4. Create a Hugging Face access token with permission to make calls to Inference Providers.

5. Create the file resume_extractor.py.

6. Run the program using: python resume_extractor.py.

7. Enter the Hugging Face token when prompted.

8. Enter the resume text and type END on a new line.

9. The program sends the resume to the AI model and displays the extracted JSON.

7. Important API Integration Code

```
client = OpenAI(
    base_url="https://router.huggingface.co/v1",
    api_key=token.strip()
)
```

The highlighted code creates an OpenAI-compatible client that sends requests to the Hugging Face router. The API token is supplied at runtime so the secret is not stored inside the source code.

8. Important Prompt Used for AI Extraction

```
prompt = f"""
You are an AI Resume Information Extractor.

Extract information from the resume below.

Return ONLY valid JSON with exactly these fields:

{{
    "candidate_name": "candidate name",
    "skills": [],
    "education": [],
    "experience": [],
    "certifications": []
}}

Rules:
- Extract only information present in the resume.
```

```
- Do not invent information.
- If a field is missing, use an empty string or empty list.
- Skills, education, experience and certifications must be arrays.
- Do not add explanations or markdown.

Resume:
{resume_text}
"""
```

The highlighted prompt is the main instruction given to the AI model. It defines the required fields, prevents invented information, and forces the response toward the required JSON structure.

9. API Request Code

```
response = client.chat.completions.create(
    model="openai/gpt-oss-120b:fastest",
    messages=[
        {
            "role": "system",
            "content": "You extract resume information and return valid JSON only."
        },
        {
            "role": "user",
            "content": prompt
        }
    ],
    temperature=0
)
```

10. Complete Python Program

```
import json
import getpass
from openai import OpenAI

print("=" * 60)
print("          AI RESUME INFORMATION EXTRACTOR")
print("=" * 60)

token = getpass.getpass("\nEnter your Hugging Face API token: ")

if not token.strip():
    print("ERROR: Token cannot be empty.")
    exit()

client = OpenAI(
    base_url="https://router.huggingface.co/v1",
    api_key=token.strip()
)

print("Hugging Face API connection initialized successfully.")

print("\n" + "=" * 60)
print("Enter your resume text.")
print("Type END on a new line when finished.")
```

```
print("=" * 60)

lines = []

while True:
    line = input()

    if line.strip().upper() == "END":
        break

    lines.append(line)

resume_text = "\n".join(lines)

if not resume_text.strip():
    print("\nERROR: Resume cannot be empty.")
    exit()

prompt = f"""
You are an AI Resume Information Extractor.

Extract information from the resume below.

Return ONLY valid JSON with exactly these fields:

{{
    "candidate_name": "candidate name",
    "skills": [],
    "education": [],
    "experience": [],
    "certifications": []
}}

Rules:
- Extract only information present in the resume.
- Do not invent information.
- If a field is missing, use an empty string or empty list.
- Skills, education, experience and certifications must be arrays.
- Do not add explanations or markdown.

Resume:
{resume_text}
"""


try:
    response = client.chat.completions.create(
        model="openai/gpt-oss-120b:fastest",
        messages=[
            {
                "role": "system",
                "content": "You extract resume information and return valid JSON only."
            },
            {
```

```
                "role": "user",
                "content": prompt
            }
        ],
        temperature=0
    )

    answer = response.choices[0].message.content.strip()

    if answer.startswith("```json"):
        answer = answer[7:]
    if answer.startswith("```"):
        answer = answer[3:]
    if answer.endswith("```"):
        answer = answer[:-3]

    result = json.loads(answer.strip())

    print("\n" + "=" * 60)
    print("              EXTRACTED INFORMATION")
    print("=" * 60)
    print(json.dumps(result, indent=4))

    print("\n" + "=" * 60)
    print("                      DONE")
    print("=" * 60)

except Exception as e:
    print("\nERROR:")
    print(e)
```

11. Sample Input

John Smith

Computer Science student with strong programming and AI skills.

Skills:
Python, C, C++, Java, Machine Learning, SQL, HTML, CSS, Git.

Education:
B.Tech in Computer Science and Engineering,
ABC Engineering College, 2024-2028.

Experience:
Python Developer Intern at XYZ Technologies for 6 months.
Worked on machine learning projects and data analysis.

Certifications:
Python for Data Science - Coursera.

Machine Learning Fundamentals - Google.

END

12. Screenshots – Actual Working Demonstration

Successful API connection and resume input screen

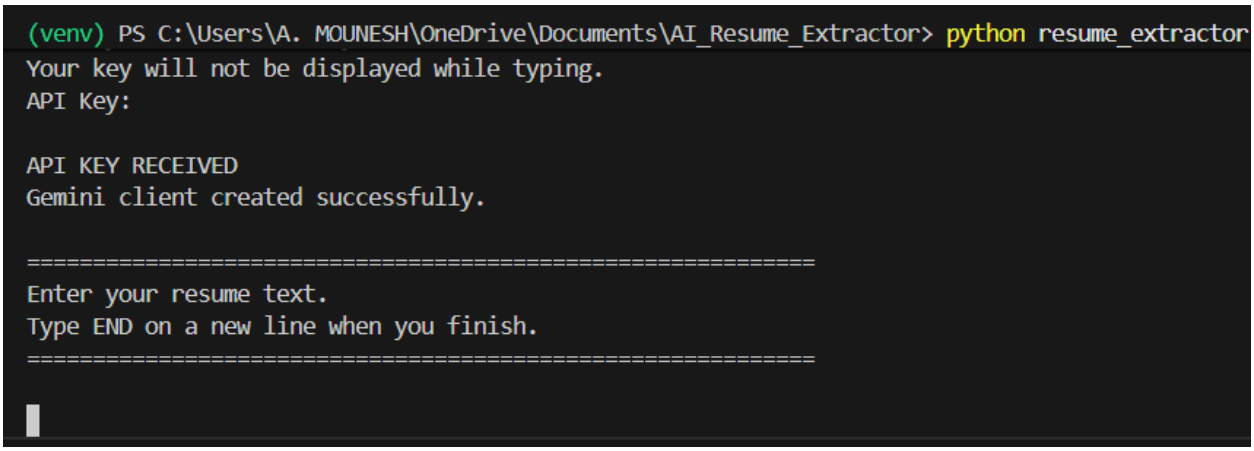

Figure: Successful API connection and resume input screen

Successful structured JSON output

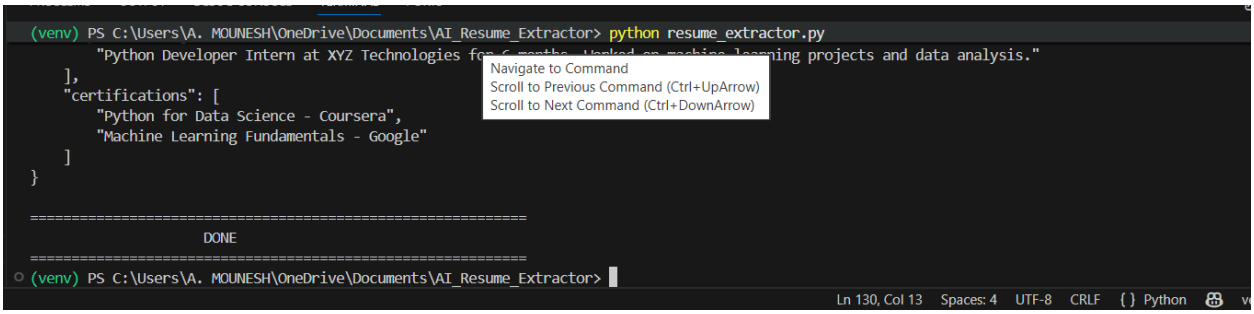

Figure: Successful structured JSON output

13. Expected / Actual Structured JSON Output

```
{
    "candidate_name": "John Smith",
    "skills": [
        "Python",
        "C",
        "C++",
        "Java",
        "Machine Learning",
        "SQL",
```

```
            "HTML",
            "CSS",
            "Git"
        ],
        "education": [
            "B.Tech in Computer Science and Engineering, ABC Engineering College, 2024-2028"
        ],
        "experience": [
            "Python Developer Intern at XYZ Technologies for 6 months",
            "Worked on machine learning projects and data analysis"
        ],
        "certifications": [
            "Python for Data Science - Coursera",
            "Machine Learning Fundamentals - Google"
        ]
    }
```

14. Edge Cases and Error Handling

- Empty API token: the program displays an error and stops safely.
- Empty resume: the program displays an error instead of sending blank input.
- Missing resume information: the prompt instructs the AI to return an empty string or empty list.
- Invalid API/request: the try-except block catches the error and displays it without crashing.

15. Modules

- Input Module – accepts multiline resume text.
- API Integration Module – connects Python to the Hugging Face API.
- Prompt Module – defines extraction instructions and output fields.
- AI Extraction Module – processes the resume using the AI model.
- JSON Processing Module – converts the response into readable JSON.
- Error Handling Module – manages invalid input and API failures.

16. Rubric Mapping

Criteria	Weightage	Evidence in This Project
API Integration Correctness	30%	Hugging Face API is integrated through the OpenAI-compatible client and a live successful run is demonstrated.
Problem-Solving Completeness	25%	All five requested resume fields are extracted into JSON.
Code Quality	20%	Structured functions, validation, secure runtime token entry and exception handling are used.
Input/Output Demo	15%	Actual VS Code screenshots show API connection, input and successful JSON output.
Code Documentation	10%	Aim, workflow, setup, prompt, code, screenshots, result and viva explanation are included.

17. Result

The AI Resume Information Extractor was successfully implemented and tested in VS Code. The application accepts unstructured resume text, sends it to the Hugging Face AI service, and produces structured JSON containing the candidate name, skills, education, experience, and certifications. The actual successful execution screenshots are included as evidence.

18. Viva / Faculty Explanation

“My project is an AI Resume Information Extractor. It accepts resume text as input and uses the Hugging Face Inference API to analyze the resume. The prompt instructs the AI to extract five fields: candidate name, skills, education, experience and certifications. The result is returned as JSON, which makes the information structured and easy to use. I also implemented validation and exception handling, and I tested the application successfully in VS Code.”

19. Final Submission Checklist

☐ Final Python source code

☐ Working API demonstration screenshot

☐ Successful JSON output screenshot

☐ Sample input

☐ Edge-case handling

☐ Rubric mapping

☐ Viva explanation